Data Mining: BeeViva Challenges

MARCO BELLÒ, University of Padua, Italy

All R source code, images and LaTeX sources, can be found at the following repository: https://github.com/mhetacc/DataMiningChallenges/. Datasets have been removed and can be found at https://www.datachallenge.it/competitions/.

CCS Concepts: • **Information systems** → **Data mining**; • **Mathematics of computing** → **Multivariate statistics**; **Statistical software**; • **Human-centered computing** → *Visual analytics.*

ACM Reference Format:
Marco Bellò. 2025. Data Mining: BeeViva Challenges. 1, 1 (December 2025), 20 pages. https://doi.org/XXXXXXX.XXXXXXX

1 RICE VARIETIES

1.1 Preliminary Observations

1.1.1 Datasets. All that follows is based on the datasets provided in the challenge page (*rice_test.csv* and *rice_train.csv*), even though they both stem from an original one, which can be found at the following link: https://www.muratkoklu.com/datasets/.

1.1.2 Scatter Plot. From the *scatter plot* (figure 1) we can infer that there are four extremely correlated features: *Area*, *Perimeter*, *Convex_Area* and *Major_Axis_Length*.

1.1.3 Correlation Matrix. I used a correlation matrix (table 1) to see exactly how correlated the features are. As suspected, all four features aforementioned have a high degree of correlation (over ninety percent). To handle this is we can either combine or remove the correlated features, or use models robust against collinearity.

1.1.4 Features' Importance. To get a better idea of the importance of each feature, I trained a simple linear regression model, and looked at the result with *summary(fit)*, which can be seen in table 2. The significance stars tell us visually that, with the exception of *Minor_Axis_Length*, highly correlated features contribute strongly to the model, while *Eccentricity* and *Extent* contribute very little.

We can also measure the total variance explained by all predictors combined using $R^2 = 0.6953849$, and we can check the standardized coefficients to better compare predictors' importance, as shown in table 3. Once again, we can see that *Extent* and *Eccentricity* contribute very little to the overall model.

1.1.5 Principal Components. A good way to aggregate and simplify data is by decomposing it into its principal components (centered in zero and scaled to have unit variance). Components' summary can be seen in table 4, and it shows us that more than 99% of overall variance can be explained with just three components, so a 2D visualization

Author's address: Marco Bellò, marco.bello.3@studenti.unipd.it, University of Padua, Padua, Italy.

Manuscript submitted to ACM

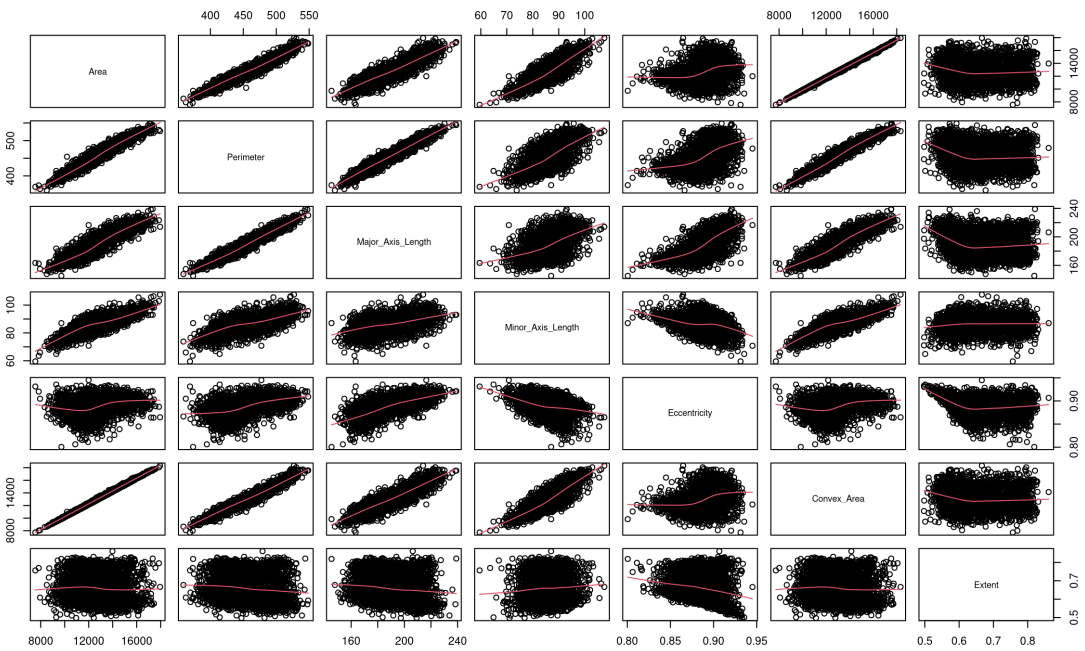


Fig. 1. Scatter Plot for the *rice_train* dataset with LOESS smoothing lines for each class.

Table 1. Correlation matrix of features

	Area	Perimeter	Major Axis Length	Minor Axis Length	Eccentricity	Convex Area	Extent
Area	1.00000000	0.96704011	0.90356325	0.79022701	0.34653177	0.99895272	-0.06002223
Perimeter	0.96704011	1.00000000	0.97163180	0.63486482	0.53784650	0.97046788	-0.12718015
Major Axis Length	0.90356325	0.97163180	1.00000000	0.45675159	0.70625660	0.90390912	-0.13562592
Minor Axis Length	0.79022701	0.63486482	0.45675159	1.00000000	-0.29393931	0.78975480	0.06192558
Eccentricity	0.34653177	0.53784650	0.70625660	-0.29393931	1.00000000	0.34707340	-0.19301157
Convex Area	0.99895272	0.97046788	0.90390912	0.78975480	0.34707340	1.00000000	-0.06397213
Extent	-0.06002223	-0.12718015	-0.13562592	0.06192558	-0.19301157	-0.06397213	1.00000000

with only the first two components should give us a good idea of the whole dataset (figure 2). The plot suggests that data is well separated, with similar classes clustering nicely. This should make classification easier.

1.2 Modeling

After exploring the dataset, I tried to find the best performing model for predicting the target.

1.2.1 Linear Regression. First I split the training dataset into training and validation sets, fitted the model and computed its mean square error: $MSE = 0.07354557$. Since we know from the preliminary observations that some features are

Table 2. Regression coefficients and coefficients' importance

Parameter	Estimate	Std. Error	t value	Pr(>|t|)	Importance
(Intercept)	-2.152e+00	1.990e+00	-1.081	0.279633	
Area	5.601e-04	1.023e-04	5.474	4.79e-08	***
Perimeter	8.440e-03	2.221e-03	3.800	0.000148	***
Major_Axis_Length	-2.198e-02	5.709e-03	-3.851	0.000120	***
Minor_Axis_Length	4.669e-02	1.213e-02	3.850	0.000121	***
Eccentricity	4.534e+00	1.828e+00	2.481	0.013170	*
Convex_Area	-8.609e-04	9.418e-05	-9.141	< 2e-16	***
Extent	7.146e-02	7.144e-02	1.000	0.317237	

Table 3. Standardized regression coefficients

Parameter	Standardized Coefficient
(Intercept)	NA
Area	1.95970669
Perimeter	0.60605955
Major_Axis_Length	-0.77272038
Minor_Axis_Length	0.54256082
Eccentricity	0.18931314
Convex_Area	-3.09099064
Extent	0.01114328

Table 4. Summary of principal components

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Standard deviation	2.2924	1.2436	0.9562	0.51393	0.10621	0.07758	0.04519	0.02052
Proportion of Variance	0.6569	0.1933	0.1143	0.03302	0.00141	0.00075	0.00026	0.00005
Cumulative Proportion	0.6569	0.8502	0.9645	0.99753	0.99894	0.99969	0.99995	1.00000

highly correlated (so the dataset suffers from collinearity), I tried to transform the dataset to reduce it. A straightforward approach is to compute the average of all of them (listing 1), but this yielded a slightly worse result: $MSE \approx 0.074$.

Another alternative would be to drop the features altogether, after measuring their variance inflation factor. As a rule of thumb, a *VIF* value above than indicates a problematic amount of collinearity, so I tried to drop features iteratively to see if the model improves, as shown in table 5. For each dropped variable I computed the *VIFs* and the *MSE* of the resulting model. As we can see, dropping *Area* resulted in performances similar to combining the variables, while dropping the other features worsened prediction performances even further.

Fig. 2. Plot with first two principal components of *rice_train* dataset.

Table 5. Effect of variable removal on variance inflation factors (VIF) and mean squared error (MSE)

Variable Dropped	VIF Perimeter	VIF Major_A_L	VIF Minor_A_L	VIF Ecc.	VIF Convex_A	VIF Extent	MSE
Area	114	206	133	53	332	1	0.0735629
Convex Area	108	134	40	49	–	1	0.07572877
Major Axis Length	47	–	37	29	–	1	0.07718435
Perimeter	–	–	1	1	–	1	0.0806868

Listing 1. Average of highly correlated features

```
rice.train$Combined <- rowMeans(rice.train[, c("Area","Perimeter","Major_Axis_Length","Convex_Area")])

linear_fit = lm(
  Class ~
  Minor_Axis_Length
  + Eccentricity
  + Extent
  + Combined,
  data=rice_train,
  subset = train
  )
```

```
pred <- predict(linear_fit, newdata = rice_train[test, ])
mean((pred - y.test)^2)     # [1] 0.07439239
```

1.2.2 Ridge Regression. I used the *glmnet* library, which takes care of the variable standardization for us and has a built-in cross-validation functionality to compute the best possible λ (code 2). The resulting model was slightly worse than linear regression, since $MSE = 0.07924703$. That being said, we can see in plot 3 an effective shrinkage of most coefficients towards zero, with the exception of *Eccentricity*, which we already recognized as not very impactful on the overall variance and prediction performances.

Listing 2. Ridge regression on rice dataset

```
library(glmnet)
cv.out <- cv.glmnet(x[train, ], y[train], alpha = 0)
plot(cv.out)
bestlam <- cv.out$lambda.min
cv_mse <- min(cv.out$cvm)     # [1] 0.07924703
cv_rmse <- sqrt(cv_mse)       # [1] 0.2815085
```

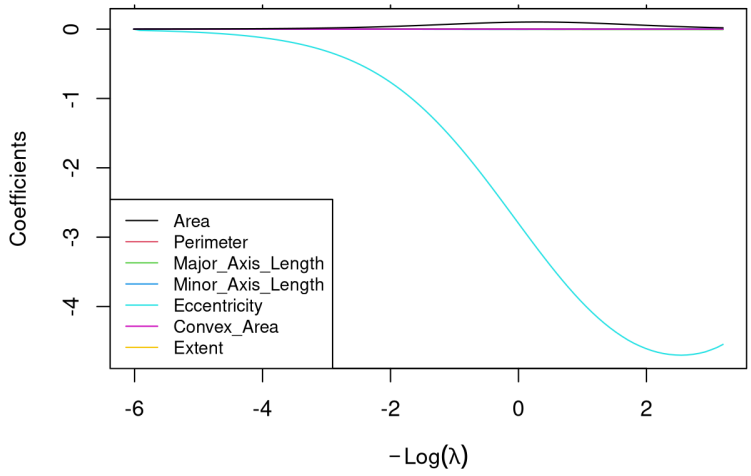


Fig. 3. Ridge coefficients for *rice_train* dataset.

1.2.3 Lasso Regression. Very similar to ridge regression, except *glmnet* gets called with *alpha = 1*. We get a slightly better result than ridge, with $MSE = 0.07348723$. From the plot (in figure 4), we can see that once again *Eccentricity* seems to resist shrinkage, but overall coefficient reduction is much more accentuated compared to ridge, since lasso can set them to zero.

1.2.4 Principal Components Regression. I fitted the model with *pls* library's built-in cross-validation, as seen in code 3. The lowest error (figure 5) is achieved with seven components: $MSE = 0.07354557$, slightly worse than lasso regression.

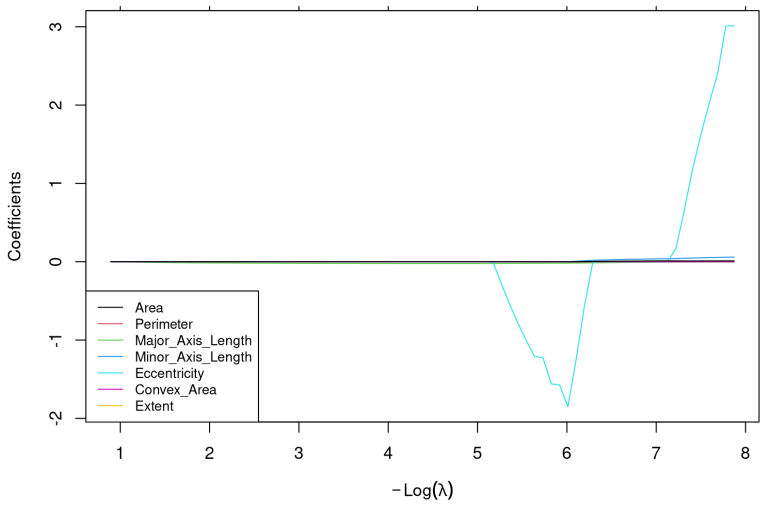


Fig. 4. Lasso coefficients for *rice_train* dataset.

Listing 3. Principal components regression on rice dataset

```
pcr.fit <- pcr(
  Class ~ .,
  data = rice_train,
  subset = train,
  scale = TRUE,
  validation = "CV"
  )
```

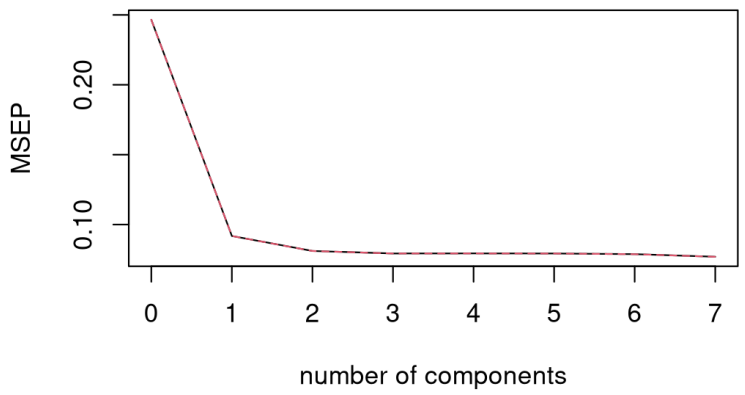


Fig. 5. PCR cross-validation MSE error for target *Class* on *rice_train* dataset.

1.2.5 Partial Least Squares. I used the same *pls* library to fit the partial least squares model on the dataset. The best result is obtained with just two components (figure 6), yielding $MSE = 0.07609556$, which is slightly worse than *PCR* but needs a third of the components, potentially leading to less overfitting and computational cost.

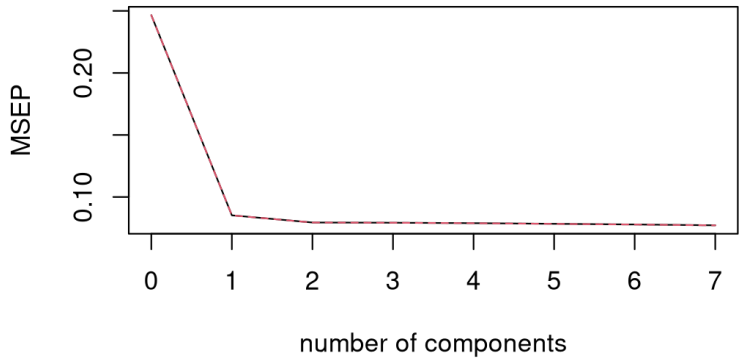


Fig. 6. PLS cross-validation MSE error for target *Class* on *rice_train* dataset.

1.2.6 LOESS. I then tried some non-parametric methods, starting with *LOESS* regression. This method allows up to four predictors, so I used the most useful ones previously identified: *Area*, *Perimeter*, *Convex_Area* and *Major_Axis_Length*. Predictors are normalized by default. Then, I fitted three models with three different values for *span*. Predictions must be sanitized since some test points can fall outside the local neighborhood (meaning being *NA*). With $span = 0.1$ we have five such predictions. The code can be seen at 4. The results were disastrous ($MSE \approx 46$), so I tried to include only two predictors. This yielded the best result yet: with $span = 0.1$, $MSE = 0.06099749$. All results are summarized in table 6

Listing 4. LOESS regression on rice dataset with four predictors and $span = 1$

```r
loess_fit1 <- loess(
  Class ~ Area
  + Perimeter
  + Convex_Area
  + Major_Axis_Length,
  data = rice_train,
  subset = train,
  span = 0.1
  )

pred1 <- predict(loess_fit1, newdata = rice_train[test, ])
mean((pred1 - y.test)^2, na.rm = TRUE)    # [1] 46.0226
```

1.2.7 K-Nearest Neighbors. I used the *caret* library to fit *KNN* both for classification and regression. The latter to get a comparable prediction with the other models, the former to be more logically sound, since *Class* is a categorical target. The code for classification can be seen at 5, between the two methods the only difference is the *metric* parameter: *"Accuracy"* for classification, *"RMSE"* for regression. For both methods I used Leave-One-Out Cross-Validation, and both RMSE plots can be seen in figure 7.

Table 6. MSE for LOESS with different *span* values and number of predictors

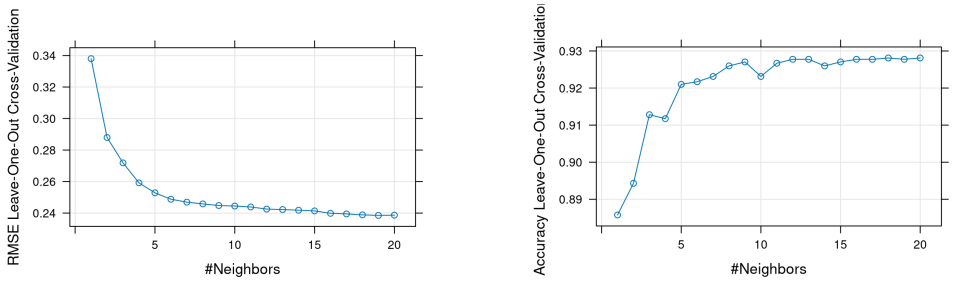
Span	Number of Predictors	MSE
0.1	Four (4)	46.0226
0.5	Four (4)	1.799422
0.9	Four (4)	0.2276324
0.1	Two (2)	0.06099749
0.5	Two (2)	0.06514592
0.9	Two (2)	0.06836381

For regression, the best result is achieved with nineteen (19) neighbors, yielding a mean square error even better than LOESS: $MSE = 0.05688406$. For classification, the best accuracy is achieved with twenty (20) neighbors, yielding an accuracy of almost 93%. This last model is the one I ultimately used to predict the target values on the test dataset.

Listing 5. KNN regression on rice dataset with Leave-One-Out Cross-Validation

```
train.control <- trainControl(method  = "LOOCV")
train$Class <- as.factor(train$Class) # necessary for classification

knn_fit <- train(
  Class ~ .,
  method     = "knn",
  tuneGrid   = expand.grid(k = 1:20),
  trControl  = train.control,
  preProcess = c("center","scale"),    # normalized
  metric     = "Accuracy",
  data       = train
  )
```



(a) RMSE of KNN for regression

(b) Accuracy of KNN for classification

Fig. 7. RMSE and Accuracy respectively of KNN for regression and for classification with *LOOCV* on *rice_train* dataset.

1.2.8 Random Forest. The last method I tried is *random forest* with Leave-One-Out Cross-Validation, using the *caret* library still. Since this method is extremely computationally expensive, I had to leverage the library *doParallel* to use all available CPU cores, and I used the faster implementation *ranger*. I also reduced the amount of predictors by combining the highly correlated ones (we already saw that dropping them yielded similar or worse results). The code can be seen at listing 6.

The optimal model, with $mtry = 2$, $splitrule = extratrees$ and $min.node.size = 5$, yielded a slightly worse result than KNN trained on all features: $MSE = 0.0589397$. The plot for the *RMSE* can be seen in figure 8

Listing 6. Random forest regression on *rice_train* dataset with Leave-One-Out Cross-Validation

```
train.control <- trainControl(method  = "LOOCV")

rf_fit <- train(
  Class ~ Minor_Axis_Length + Eccentricity + Extent + Combined,
  method     = "ranger",
  tuneLength = 10,
  trControl  = train.control,
  metric     = "RMSE",
  data       = rice_train,
  allowParallel = TRUE
)
```

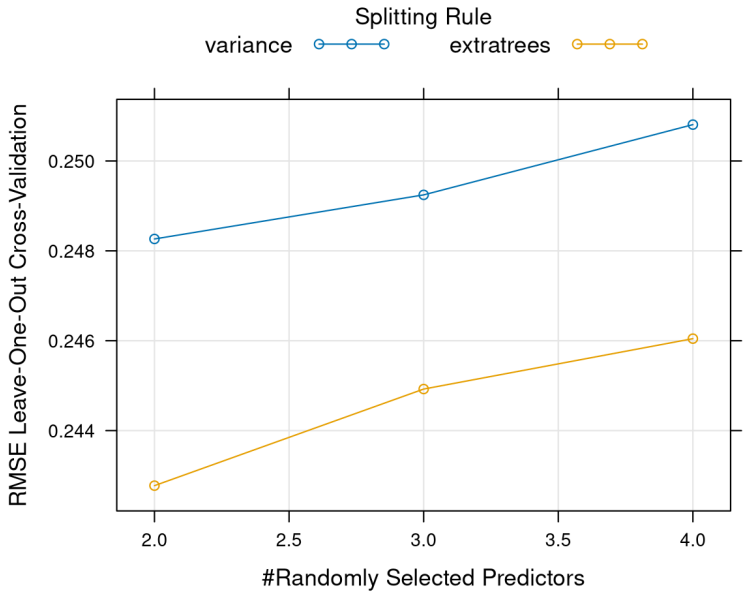


Fig. 8. RMSE error for random forest on *rice_train* dataset.

1.3 Rice Varieties Conclusions

The best model so far is KNN with $K = [19, 20]$. While slightly more computationally expensive than other methods (with the exception of random forest), it improves MSE by 0.00411343 over the second best method (LOESS with $span = 0.1$). I decided to predict the target with a KNN fit in classification mode, since I felt it was more logically sound.

I decided not to try random forest with all predictors for time constraints: it required a 10-Cores machine (i7-13620H @ 4.90 GHz) twenty-five minutes to fit the model presented in section 1.2.8.

2 PHONE USERS

2.1 Preliminary Observations

2.1.1 Dataset. Let's first briefly analyze the features of the dataset:

- **For each user:**
  - Plan type;
  - Payment method;
  - Sex;
  - Activation zone;
  - Activation channel;
  - Value-added service 1;
  - Value-added service 2;
  - **For each month:**
    - |expensive calls|;
    - |cheap calls|;
    - time(expensive calls);
    - time(cheap calls);
    - cost(expensive calls);
    - cost(cheap calls);
    - |incoming calls|;
    - time(incoming calls);
    - |SMS|;
    - |calls to call center|.

Considering that the target is monthly calls time, we can make the hypothesis that some features can be ignored, for example any monthly feature not related to calls time itself, leaving us with only $time(expensivecalls)$ and $time(cheapcalls)$, which, since overall cost is not a concern, we could further assume can be combined. It goes without saying that all of the above must be proven empirically.

2.1.2 Scatter Plot. I filtered out all non-calls-time-related monthly data. The result (figure 9) is a bit hard to read due to its sheer size. Looking at it up close yields more insights: first of all we see that calls data looks positively skewed (figure 10), and that there are some activation channels that see more calls time than others, while activation regions seem to differ little between one another (figure 11a). On the other hand, plan and sex seem to have an impact on calls time (figure 11b), while age does not seem to have a big impact, with the exception of the very young and very old (figure 11c).

2.1.3 Calls Features Over Everything Else. I then wanted to see if calls amount and time were related to other features. First I plotted total calls taken and total calls time over nine months (figure 13). We can see that the graphs are almost identical from a trend standpoint, hinting at a strong correlation between number of calls and time spent calling.

Then I wanted to see whether sex plays any role in total calls time. Using the library *dplyr* I computed all data in table 7. On average, men's calls seem to last 8% longer. To mitigate the effect of outliers I filtered out the top 1%, which still shows the same trend, albeit reduced to 5%. Cutting even further the top 10% shows a bigger increase in men's average calls times of about 10%. Lastly, I tried to log-transform the data. Even in this situation we see that men have longer calls, hinting that this predictor may be useful. I did exactly the same with payment methods, activation zones,

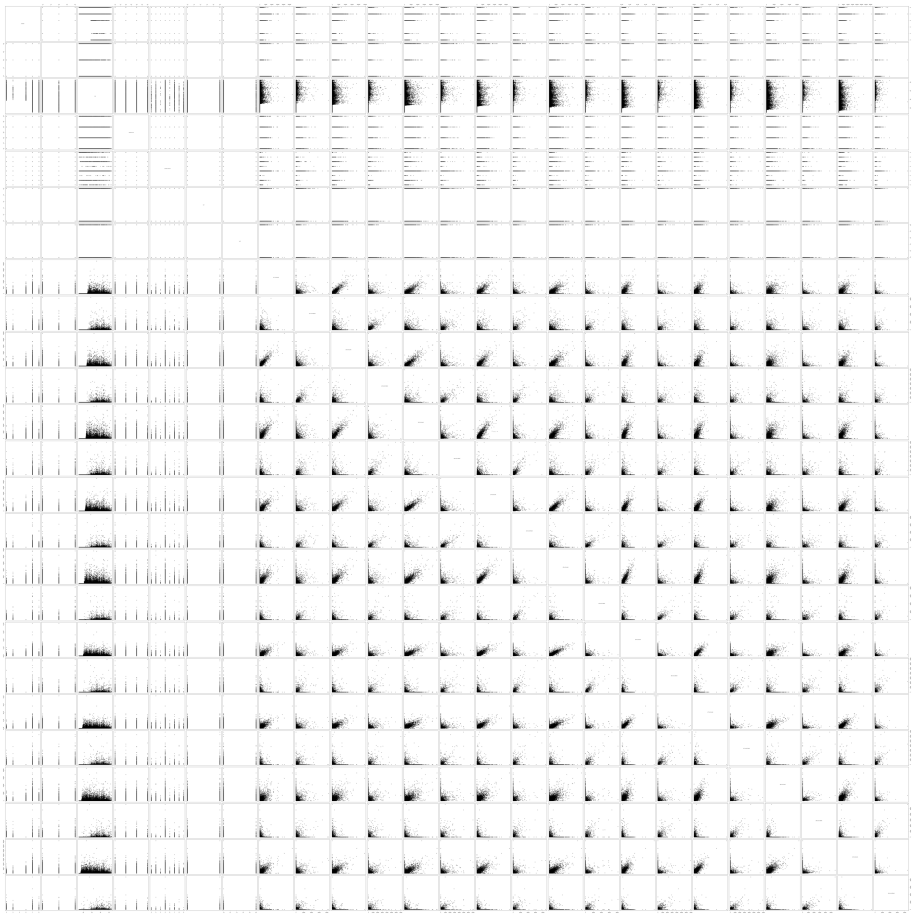


Fig. 9. Scatter Plot for the filtered *phone_train* dataset.

activation channels and both value-added services, always cutting off the top 1%. All data can be seen in table 8. There seem to be quite a bit of variance in calls time compared to the tariff plan, which would make sense: some plans may be geared toward calling, while others are more suited for sending SMS. We can see some variance on payment method too. Activation zone and activation channel seem to have some influence on calls times, with the exception of zone eight, whose calls times are way lower. On the other hand, customers that have activated either first or second added-value services seem to call for longer.

2.1.4 Skewness. calls time data is usually positively skewed, with many users having low usage and a small number of users with very high usage (e.g.,50.000 seconds). This is easily verifiable at a glance if we look at the density graph in figure 13a. This effect can be mitigated by log-transforming the data, as shown in figure 13b. Moreover, with the *e1071* library I computed a skewness matrix for all features (appendix A.1): all monthly-based data (calls, calls times, SMS, costs, etc) is highly positively skewed. To manage this problem we can either transform it (log, square root or Box-Cox

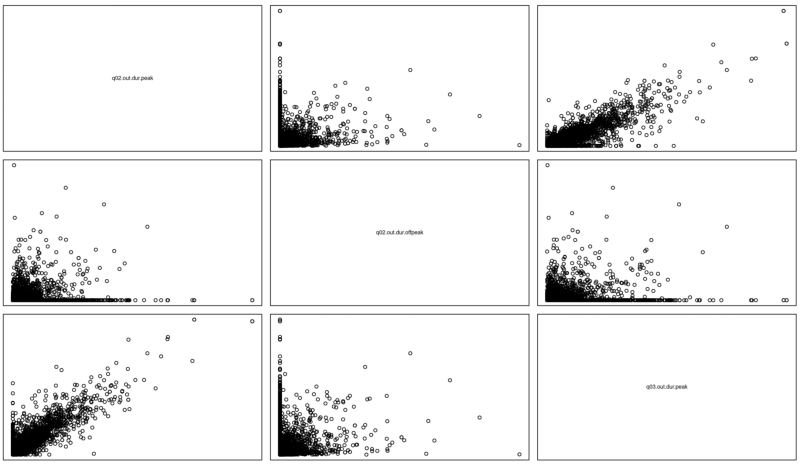
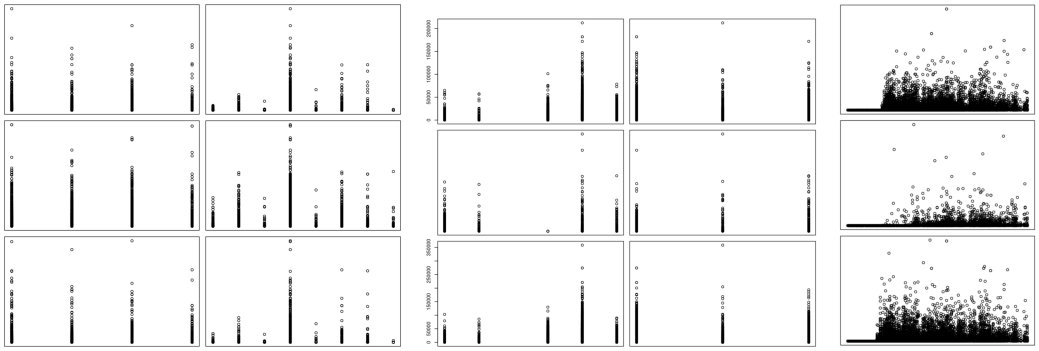


Fig. 10. Scatter Plot for calls time for *phone_train* dataset.



(a) Activation zone (X-axis, L) and activation channel (X-axis, R) over calls time (Y axis)

(b) Plan type (X-axis, L) and sex (X-axis, R) over calls time (Y axis)

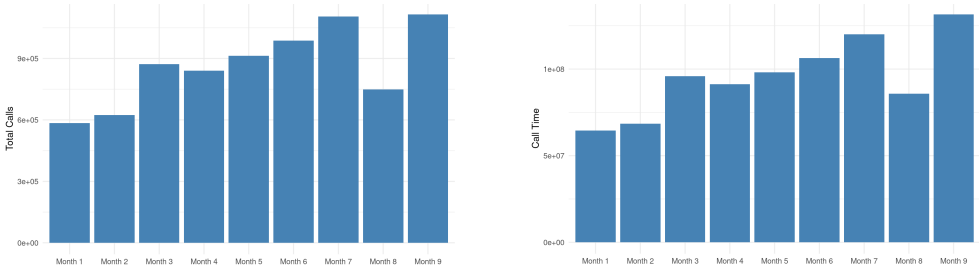
(c) Age (X-axis) over calls time (Y-axis)

Fig. 11. Different features plotted over calls time for the *phone_train* dataset.

transform), or use more robust models (such as random forest, Gamma/Poisson or quantile regression). On the other hand, methods like linear, ridge or lasso regression, and even some non-parametric ones like KNN are not ideal.

2.1.5 Correlation Matrix. Since the *cor* function only works on numerical data, I transformed all categorical data with the *model.matrix* function. We can infer the following: age is correlated to all calls-related monthly features by roughly 25%, first and second value-added services are correlated to some, not all, monthly features by roughly 10%, and the only features that share significant correlation are the monthly calls-related ones. For example, in month four outgoing calls time and calls expenses share a correlation of 97%, while for the seventh month the correlation drop to 57%, which is still significant.

2.1.6 Linear Regression Insights. A preliminary analysis can be done by plotting a linear regression fit (figure 14a). We can see that the data presents high heteroskedasticity, skewness and in general it is not normally distributed. There are

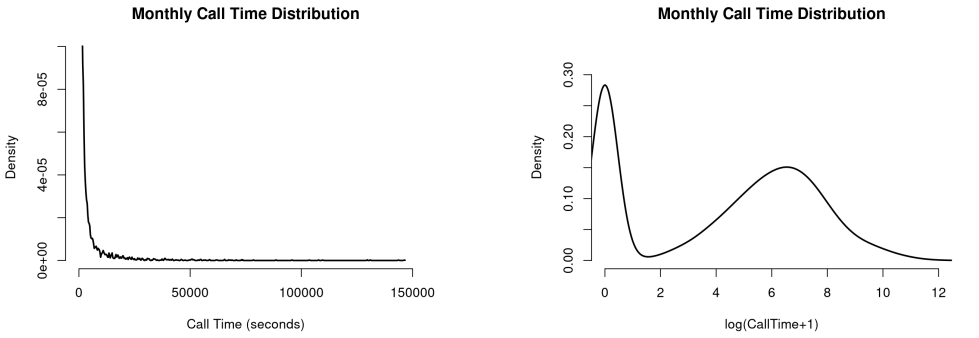


(a) Total amount of calls per month (b) Total calls time per month

Fig. 12. Total calls and total calls time per month for the *phone_train* dataset.

Table 7. calls time statistics by sex under different data cuts and transformations

Type of cut	Sex	Number of customers	Total calls time	Average calls time
None (raw)	B	5266	557266412	105823
None (raw)	F	1030	62493290	60673
None (raw)	M	3704	242226228	65396
Top 1% cut	B	5201	498846876	95914
Top 1% cut	F	1025	57203392	55808
Top 1% cut	M	3674	215801665	58738
Top 10% cut	B	4595	276063673	60079
Top 10% cut	F	960	32400501	33751
Top 10% cut	M	3445	130747836	37953
Log-transformed	B	5266	557266412	10.00
Log-transformed	F	1030	62493290	8.24
Log-transformed	M	3704	242226228	8.58



(a) calls time density distribution (b) calls time density distribution (log-transformed)

Fig. 13. calls time density distributions for the *phone_train* dataset.

Table 8. calls time over different categorical features

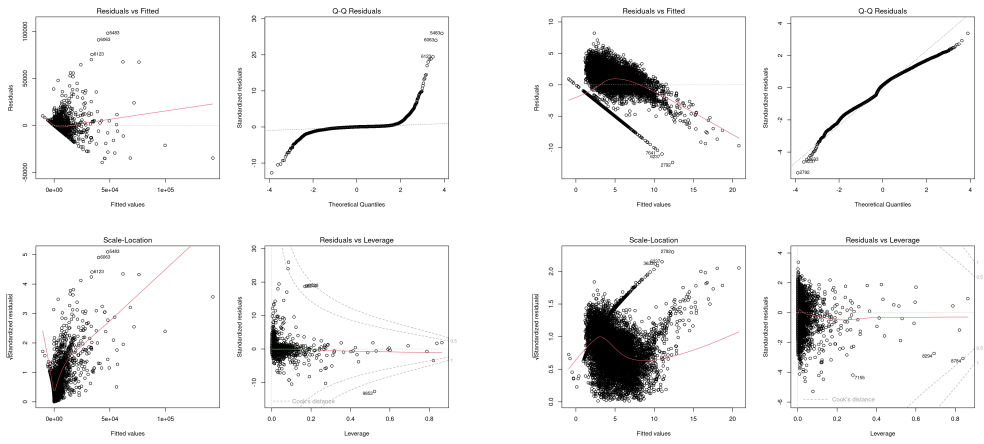
Feature	Number of customers	Total calls time	Average calls time
Tariff plan 3	781	44811041	57376
Tariff plan 4	83	10575009	127410
Tariff plan 6	724	92214268	127368
Tariff plan 7	3564	489635465	137384
Tariff plan 8	4748	134616150	28352
Activation zone1	3507	288670845	82313
Activation zone2	3130	213438509	68191
Activation zone3	2342	197941555	84518
Activation zone4	921	71801024	77960
Activation channel 2	130	10643060	81870
Activation channel 3	408	36663577	89862
Activation channel 4	31	2369822	76446
Activation channel 5	7135	623961079	87451
Activation channel 6	47	4352853	92614
Activation channel 7	652	62777153	96284
Activation channel 8	111	12074180	108776
Activation channel 9	1386	19010209	13716
Value-added one: No	7450	526620961	70687
Value-added one: Yes	2450	245230972	100094
Value-added two: No	9279	695404226	74944
Value-added two: Yes	621	76447707	123104

also potential outliers and influential points that could distort the model, such as point 9853. As for features relevance (appendix A.2), for the most part, relevant features are the ones related to monthly calls. It is interesting to notice that the second month does not seem to count much, although having a similar amount of calls data compared to the others.

Log-transforming the target variable yielded better results (figure 14b), but the above-mentioned problems did not disappear. Feature relevance held some surprises (appendix A.3), for example categorical features contributed much more to the model and only a small subset of monthly calls data was relevant.

2.1.7 Principal Components. Data as-is can be transformed into 101 principal components, and the cumulative proportion with only two is approximately 0.45. To get to 99% of explained variance we need 69 components. Plotting the first two components shows us, once again, signs of heteroskedasticity and non-normal data distribution, as shown in figure 15

2.1.8 Preliminary Observations Conclusions. Data shows high skewness and heteroskedasticity, and some features do not seem to contribute much to the model. We probably should try to filter out some data, for example SMS amount, transform monthly calls data, for example with log or Box-Cox transforms, and use some robust regression methods, such as quantile or tree-based approaches.



(a) Linear regression fit plot

(b) Linear regression fit plot (log-transformed)

Fig. 14. Linear regression fit plots for *phone_train* dataset.

2.2 Modeling

2.2.1 Lasso Regression. I used lasso regression with cross-validation as a preliminary diagnostic tool to check which features are better off dropped or transformed. First, I made a baseline prediction without transforming any value, with the following mean square error: $MSE = 18091358$. Not great, but expected since we saw in the preliminary observations many signs of non-linearity.

I then tried log-transforming the target value (with all features), which resulted in a massive improvement: $MSE = 6.200236$.

Then, on the not transformed target, I tried different predictions with different data filtering, with the following results: filtering out incoming calls, SMS and calls to the call center yielded a 3% improvement over the all-features model ($MSE = 18040253$). Putting calls to the call center back in worsened the model slightly ($MSE = 18080968$). I thus decided to leave these features filtered out permanently.

Then, I tried to filter out either both or any between calls value and calls amount, which produced a worse performing model in all instances. Then, I filtered out non-monthly features, and discovered in the end that the best result ($MSE = 18019454$) can be obtained with the following features **filtered out**: payment method, value-added service two, activation zone and activation channel (plus, of course, the previously mentioned incoming calls, SMS and calls to the call center).

I then tried to pre-process the monthly features, such as adding together peak and off-peak values or predict on the average monthly calls times. All yielded worse results.

Lastly, with the best performing filtering, I log-transformed only the target ($MSE = 6.253648$), only the monthly features ($MSE = 20415157$), and then both, which ultimately yielded the best result: $MSE = 4.289544$. It is worth noting that log-transforming the target on the filtered dataset produced a worse performing fit than a model fitted on all data and log-transformed target.

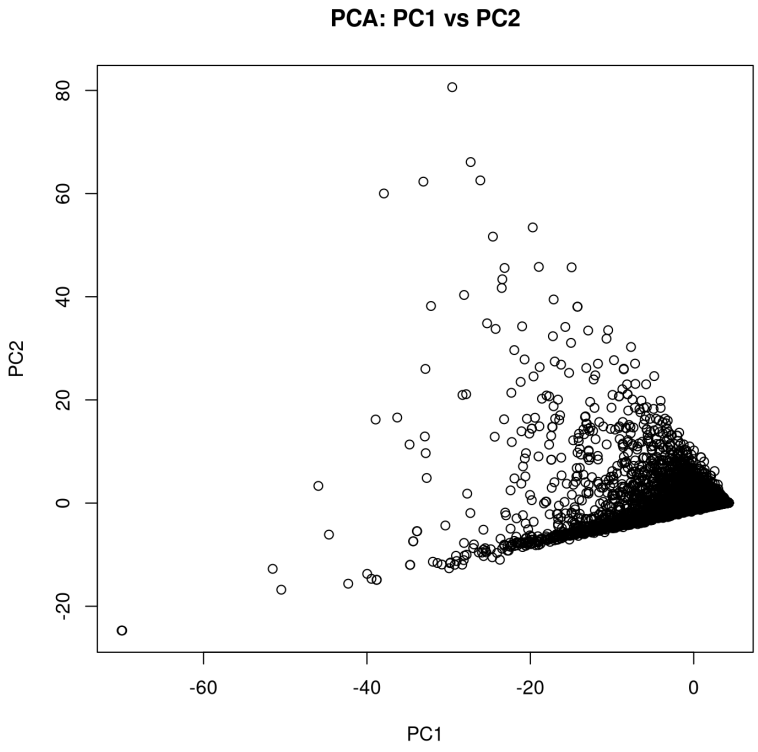


Fig. 15. Scatter Plot for calls time for the *phone_train* dataset.

To summarize: the best performing model is obtained by filtering out payment method, value-added service two, activation zone, activation channel, incoming calls, SMS and calls to the call center, and by log-transforming the target and the remaining monthly calls-related features.

The complete list of filtering and transformations can be seen in appendix A.4

2.2.2 Random Forest. Considering all of the above, a model inherently robust to non-normally distributed data should yield better results. random forest is a good example of such a model. I still log-transformed both the target and the predictors since it showed good results. I fitted the model with the *ranger* method on a *tuneLength* of ten with cross-validation. Compared to the previous challenge I had to give up on LOOCV due to memory constraints. The resulting best fit (figure 16,with $mtry = 8$, variance as the split rule and minimum node size of five) yielded the best mean square error overall: $MSE = 3.947784$, which is why I ultimately used this model to predict the target value.

2.3 Phone Users Conclusions

The dataset was, in fact, not normally distributed and non-linear, so transforming the features greatly improved a linear fit such as lasso, and using a robust model yielded even better results.

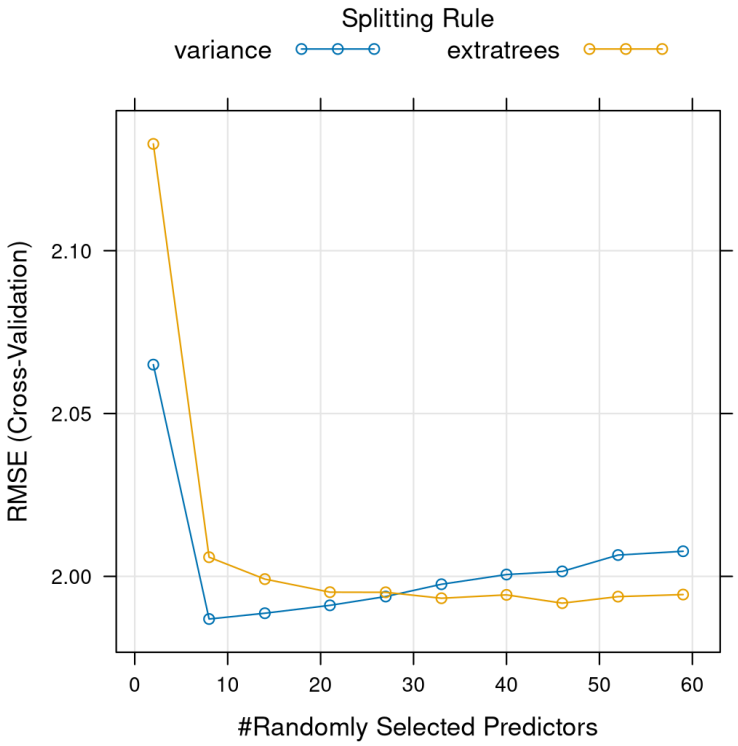


Fig. 16. RMSE for random forest for the *phone_train* dataset.

A APPENDICES

A.1 Phone call time skewness

Listing 7. Skweness matrix

```
        tariff.plan  activation.channel     q01.out.ch.peak    q01.out.dur.peak    q01.out.val.peak
          -2.026605            1.073333            4.930308            5.883900           10.158610
q01.out.dur.offpeak q01.out.val.offpeak       q01.in.ch.tot      q01.in.dur.tot          q01.ch.sms
          9.336163           31.977903            4.322467            3.615376           37.220847
    q02.out.ch.peak    q02.out.dur.peak    q02.out.val.peak  q02.out.ch.offpeak q02.out.dur.offpeak
          4.011558            5.026925            6.120357            9.636244            9.202850
      q02.in.ch.tot      q02.in.dur.tot          q02.ch.sms           q02.ch.cc     q03.out.ch.peak
          3.657302            3.444048           38.464828           10.697464            3.742071
   q03.out.val.peak  q03.out.ch.offpeak q03.out.dur.offpeak q03.out.val.offpeak       q03.in.ch.tot
          4.144170            8.709344            9.948054           25.074765            3.359255
         q03.ch.sms           q03.ch.cc     q04.out.ch.peak    q04.out.dur.peak    q04.out.val.peak
         35.231226           12.146903            3.726583            4.415209            3.988589
q04.out.dur.offpeak q04.out.val.offpeak       q04.in.ch.tot      q04.in.dur.tot          q04.ch.sms
         12.366396           10.517232            3.362859            3.660378           33.513288
    q05.out.ch.peak    q05.out.dur.peak    q05.out.val.peak  q05.out.ch.offpeak q05.out.dur.offpeak
          3.578910            4.056863            4.242749           12.129119           12.598793
```

```
      q05.in.ch.tot       q05.in.dur.tot           q05.ch.sms           q05.ch.cc      q06.out.ch.peak
           3.450009             3.165951            38.804264             9.085293             3.364608
   q06.out.val.peak  q06.out.ch.offpeak q06.out.dur.offpeak q06.out.val.offpeak       q06.in.ch.tot
           4.037063            15.035554            12.511158            10.149751             3.386507
         q06.ch.sms           q06.ch.cc     q07.out.ch.peak    q07.out.dur.peak    q07.out.val.peak
          28.668984            13.747535             3.157588             4.088846             3.830199
q07.out.dur.offpeak q07.out.val.offpeak       q07.in.ch.tot      q07.in.dur.tot           q07.ch.sms
          12.907377            10.128761             3.237636             3.192192            28.628610
    q08.out.ch.peak    q08.out.dur.peak    q08.out.val.peak  q08.out.ch.offpeak q08.out.dur.offpeak
           3.963771             4.131527             4.117853             9.256204            11.536043
      q08.in.ch.tot       q08.in.dur.tot           q08.ch.sms           q08.ch.cc      q09.out.ch.peak
           3.972012             3.953185            20.317293             8.946827             2.927636
   q09.out.val.peak  q09.out.ch.offpeak q09.out.dur.offpeak q09.out.val.offpeak       q09.in.ch.tot
           3.646469            10.344301            15.974234            11.674903             2.728376
         q09.ch.sms           q09.ch.cc                    y
          16.048910            13.034136            10.820427
```

A.2 Linear Regression Features Relevance

Listing 8. Linear regression features relevance

```
                                  t value Pr(>|t|)
(Intercept)                        15.136  < 2e-16 ***
tariff.plan                       -23.320  < 2e-16 ***
payment.methodDomiciliazione Bancaria   2.029 0.042513 *
age                                -3.290 0.001004 **
activation.channel                  2.445 0.014485 *
vas1Y                               2.637 0.008389 **
q01.out.ch.peak                    -2.998 0.002725 **
q01.out.dur.offpeak                 3.006 0.002652 **
q01.out.val.offpeak                -6.760 1.46e-11 ***
q03.out.val.peak                    3.534 0.000411 ***
q03.in.dur.tot                     -4.009 6.15e-05 ***
q04.out.dur.peak                   -3.183 0.001460 **
q04.out.val.peak                    2.320 0.020335 *
q04.out.ch.offpeak                 -4.137 3.55e-05 ***
q05.out.ch.offpeak                  2.194 0.028282 *
q05.out.dur.offpeak                -2.375 0.017556 *
q05.out.val.offpeak                 3.723 0.000198 ***
q05.in.ch.tot                      -3.401 0.000674 ***
q05.in.dur.tot                      2.231 0.025735 *
q06.out.dur.peak                   -3.182 0.001469 **
q06.out.val.peak                    2.791 0.005260 **
q06.out.ch.offpeak                 -4.226 2.40e-05 ***
q06.out.val.offpeak                 4.772 1.85e-06 ***
q06.in.dur.tot                      2.642 0.008249 **
q07.out.dur.peak                    3.250 0.001160 **
q07.in.ch.tot                       4.881 1.07e-06 ***
q07.in.dur.tot                     -4.645 3.45e-06 ***
q08.out.ch.peak                     2.856 0.004293 **
q08.out.val.peak                   -4.434 9.33e-06 ***
q08.out.ch.offpeak                 -4.099 4.19e-05 ***
q08.out.dur.offpeak                 4.495 7.04e-06 ***
q08.out.val.offpeak                 2.112 0.034720 *
q09.out.ch.peak                    -6.673 2.64e-11 ***
```

```
q09.out.dur.peak                        -0.825 0.409148
q09.out.val.peak                        11.179  < 2e-16 ***
q09.out.ch.offpeak                      10.026  < 2e-16 ***
q09.out.dur.offpeak                     18.656  < 2e-16 ***
q09.out.val.offpeak                     -4.800 1.61e-06 ***
q09.ch.cc                               -2.097 0.036019 *
```

A.3 Linear Regression Log-Transformed Features Relevance

Listing 9. Linear regression log-transformed features relevance

```
                                    Pr(>|t|)
(Intercept)                          < 2e-16 ***
tariff.plan                          < 2e-16 ***
payment.methodCarta di Credito      0.007256 **
payment.methodDomiciliazione Bancaria 0.001023 **
sexF                                0.000623 ***
sexM                                6.81e-09 ***
age                                  < 2e-16 ***
activation.channel                  4.42e-07 ***
vas1Y                               2.43e-10 ***
q01.out.ch.peak                     0.004972 **
q01.in.dur.tot                      0.009498 **
q03.out.ch.peak                     0.016320 *
q04.out.dur.peak                    0.002052 **
q04.out.val.peak                    0.012237 *
q04.out.ch.offpeak                  0.031593 *
q04.ch.cc                           0.036098 *
q05.in.ch.tot                       0.041127 *
q06.out.val.offpeak                 0.032902 *
q07.out.dur.offpeak                 0.014165 *
q07.out.val.offpeak                 0.004209 **
q08.out.ch.peak                     0.009375 **
q09.out.ch.peak                      < 2e-16 ***
q09.out.val.peak                    0.000828 ***
q09.out.ch.offpeak                  3.27e-08 ***
q09.in.ch.tot                       0.000294 ***
q09.ch.sms                          0.001544 **
```

A.4 Lasso Regression Progressive Filtering And Transformations Results

- No filtering: MSE = 18091358;
- Only log-transform target:MSE = 6.200236;
- Filtered incoming calls, SMS and calls to the call center: MSE = 18040253;
- Filtered incoming calls, SMS: MSE = 18080968;
- Only Value of calls filtered out: MSE = 18067089;
- Only amount of calls filtered out: MSE = 18118489;
- Incoming calls, SMS, call-center calls, and payment information filtered out: MSE = 18035221;
- Incoming calls, SMS, call-center calls, payment information, and sex filtered out: MSE = 18036839;
- Incoming calls, SMS, call-center calls, payment information, sex, activation zone, and activation channel filtered out: MSE = 18034269;

- Incoming calls, SMS, call-center calls, payment information, sex, activation zone, activation channel, and age filtered out: $MSE = 18034610$;
- Incoming calls, SMS, call-center calls, payment information, sex, activation zone, activation channel, age, and plan information filtered out: $MSE = 18975290$;
- Incoming calls, SMS, call-center calls, payment information, sex, activation zone, activation channel, age, plan information, and value-added services filtered out: $MSE = 18969265$;
- Incoming calls, SMS, call-center calls, payment information, value-added services, activation zone, and activation channel filtered out: $MSE = 18021270$;
- Incoming calls, SMS, call-center calls, payment information, value-added services, and activation zone filtered out: $MSE = 18022544$;
- Incoming calls, SMS, call-center calls, payment information, value-added services, and activation channel filtered out: $MSE = 18024845$;
- Incoming calls, SMS, call-center calls, payment information, value-added service 1, activation zone, and activation channel filtered out: $MSE = 18035228$;
- Incoming calls, SMS, call-center calls, payment information, value-added service 2, activation zone, and activation channel filtered out: **MSE = 18019454 (best)**.

Transformations:

- Peak and off-peak values aggregated together: $MSE = 32024591$;
- Peak and off-peak values aggregated together, with prediction performed on average call time, number of calls, and value over nine months: $MSE = 32174039$.

With best filtering:

- Log-transform of the target variable only: $MSE = 6.253648$;
- Log-transform of the monthly predictors only: $MSE = 20415157$;
- Log-transform of both the target variable and the monthly predictors: $MSE = 4.289544$.

Received 28/12/2025